

To build a highly specialized Kotlin Android architecture that functions as a persistent background processing node using continuous integration, specific system APIs and lifecycle frameworks must be coupled together.

Because this application relies on system-level capabilities, it must be signed with a matching platform signature or given deep elevated permissions to prevent Android's low-memory killer (LMK) from terminating the daemon context.

1. The GitHub CI Workflow (.github/workflows/android-ci.yml) [1]

This GitHub Actions blueprint automates your pipeline. It checks out your code, provisions JDK 21, ensures execution rights for Gradle, compiles your custom APK, and structures the artifact output. [1, 2]

name: Build System Daemon APK

on:

push:

branches: ["main", "dev"]

pull_request:

branches: ["main"]

jobs:

build:

runs-on: ubuntu-latest

steps:

- name: Checkout Code Base

uses: actions/checkout@v4

- name: Set up JDK 21

uses: actions/setup-java@v4

with:

distribution: 'temurin'

java-version: '21'

cache: 'gradle'

- name: Grant Execute Permission for Gradle Wrapper

run: chmod +x gradlew

- name: Run Code Linters and Unit Tests

run: ./gradlew test

- name: Build Elevated Release APK

run: ./gradlew assembleRelease

- name: Upload Compiled Daemon Package

uses: actions/upload-artifact@v4

with:

name: Daemon-Engine-Release

path: app/build/outputs/apk/release/app-release.apk

2. Android Manifest Configuration (AndroidManifest.xml)

To act as a Daemon, a Device Assistant API provider, and a registered Video Game, your manifest requires structural definitions that declare these operational hooks to the Android system.

```
<manifest xmlns:android="
```

```
http://android.com](https://www.android.com
```

```
"
```

```
package="com.example.systemdaemon">
```

```
<!-- Permissions required for continuous execution and system level oversight -->
```

```
<uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
```

```
<uses-permission
```

```
android:name="android.permission.FOREGROUND_SERVICE_SPECIAL_USE" />
```

```
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />
```

```
<uses-permission
```

```
android:name="android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS"/>
```

```
<!-- Large heap minimizes sudden LMK kills during on-device model orchestration -->
```

```
<application
```

```
android:allowBackup="false"
```

```
android:largeHeap="true"
```

```
android:label="@string/app_name">
```

```
<!-- 1. Video Game Registration Hook -->
```

```
<meta-data android:name="android.game.category" android:value="true" />
```

```
<!-- 2. Device Assistant API Configuration -->
```

```
<service
```

```
android:name=".services.SystemAssistantVoiceService"
```

```
android:label="HTP Handoff Assistant"
```

```
android:permission="android.permission.BIND_VOICE_INTERACTION"
```

```
android:exported="true">
```

```
<meta-data
```

```
android:name="android.voice_interaction"
```

```
android:resource="@xml/assistant_interaction_info" />
```

```
<intent-filter>
```

```

        <action android:name="android.service.voice.VoiceInteractionService" />
    </intent-filter>
</service>

<!-- 3. Low-Level Persistent Daemon Service -->
<service
    android:name=".services.DaemonProcessingEngine"
    android:foregroundServiceType="specialUse"
    android:exported="false" />

<!-- Boot receiver to spin up daemon immediately on device activation -->
<receiver android:name=".receivers.BootReceiver" android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.BOOT_COMPLETED" />
    </intent-filter>
</receiver>

</application>
</manifest>

```

[3]

Note: For the Assistant component to bind properly, you must create a resource file at `res/xml/assistant_interaction_info.xml` containing:

```

<voice-interaction-service xmlns:android="
http://android.com

    android:sessionService="com.example.systemdaemon.services.AssistantSessionService"

    android:recognitionService="com.example.systemdaemon.services.AssistantRecognitionService"
    android:supportsAssist="true"
    android:supportsLocalInteraction="true" />

```

3. The Kotlin Daemon implementation (DaemonProcessingEngine.kt)

True system daemons in standard Linux do not exist within Android's sandbox application lifecycle. Instead, you must instantiate a Sticky Foreground Service bound to an unbreakable system notification channel to replicate daemon behavior.

```
package com.example.systemdaemon.services
```

```

import android.app.Notification
import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.Service
import android.content.Intent
import android.os.IBinder
import androidx.core.app.NotificationCompat
import kotlinx.coroutines.*

class DaemonProcessingEngine : Service() {

    private val serviceJob = Job()
    private val serviceScope = CoroutineScope(Dispatchers.Default + serviceJob)
    private val CHANNEL_ID = "system_daemon_channel"

    override fun onCreate() {
        super.onCreate()
        createNotificationChannel()
        startForeground(1001, buildDaemonNotification(), Service.START_STICKY)
    }

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        // Spin up perpetual asynchronous processing loop
        serviceScope.launch {
            while (isActive) {
                // Execute low level operations / monitor Hexagon HTP pipeline states
                delay(5000)
            }
        }

        // START_STICKY instructs OS to recreate service if evicted under resource pressure
        return START_STICKY
    }

    override fun onDestroy() {
        super.onDestroy()
        serviceJob.cancel() // Safeguard memory context leaks
    }

    override fun onBind(intent: Intent?): IBinder? = null

    private fun createNotificationChannel() {
        val channel = NotificationChannel(
            CHANNEL_ID, "System Daemon Engine",
            NotificationManager.IMPORTANCE_MIN
        ).apply { description = "Maintains persistent pipeline hardware states." }

        val manager = getSystemService(NotificationManager::class.java)
        manager?.createNotificationChannel(channel)
    }
}

```

```

private fun buildDaemonNotification(): Notification {
    return NotificationCompat.Builder(this, CHANNEL_ID)
        .setContentTitle("Daemon Engine Active")
        .setContentText("Monitoring dedicated HTP compute layers...")
        .setPriority(NotificationCompat.PRIORITY_MIN)
        .build()
}
}

```

4. Integrating the Device Assistant API (SystemAssistantVoiceService.kt)

To access screen structures, system contexts, or globally intercept user assist long-press triggers (like an alternative to Google Assistant or Bixby), your app implements VoiceInteractionService. [4]

```

package com.example.systemdaemon.services
import android.os.Bundle
import android.service.voice.VoiceInteractionService
class SystemAssistantVoiceService : VoiceInteractionService() {

    override fun onReady() {
        super.onReady()
        // Assistant architecture fully mapped into the system lifecycle
    }

    override fun launchVoiceAssistFromKeyguard() {
        super.launchVoiceAssistFromKeyguard()
        // Custom processing routing handled when device is locked
    }
}

```

🛡️ Elevated Permissions Validation

Because this app is acting as a raw system resource agent, standard runtime user popups are insufficient. To elevate execution permissions, pass these configuration steps via an Android Debug Bridge (adb) terminal thread post-deployment:

1. Bypass Doze Engine / Battery Optimization Limits:

```
adb shell dumpsys deviceidle whitelist +com.example.systemdaemon
```

2. Manually Force-Bind Device Assistant Context Default Routing:

```
adb shell settings put secure assistant
com.example.systemdaemon/.services.SystemAssistantVoiceService
```

Would you like assistance setting up the boilerplate code for the associated AssistantSessionService and AssistantRecognitionService required to capture screen text data through this custom Assistant API layer?

[1] (<https://www.youtube.com/watch?v=9CYbB39PtSs>)

[2]

(<https://github.com/marketplace/actions/automated-build-android-app-with-github-action>)

[3] nu([https](https://www.samsung.com/us/support/answer/ANS10001575/))

(<https://www.samsung.com/us/support/answer/ANS10001575/>)

This setup creates a robust, self-healing system pipeline. To achieve automated model swapping, remote updates, terminal orchestration, and cross-application persistence on Android, the system must bypass the typical restrictions of the application sandbox.

1. Dual-App Watchdog System (The Ghost Reciprocal Recovery Pattern)

Android will aggressively kill heavy background tasks like your 9B LLM processing node if system memory runs low. To prevent this, implement two distinct applications with opposing broadcast hooks that monitor and revive each other completely out of band.

[App A: Main Engine]	[App B: Watchdog Engine]
(Package: com.sys.daemon)	(Package: com.sys.watchdog)

```

|                                     |
| CRASH / OOM EVICTION               |
|====[Broadcast: ACTION_PACKAGE_REPLACED]====>
|                                     |
|                                     | (Wakes up)
|                                     | Launches Service
|
|<==[Intent: startForegroundService()]--|
```

App A Manifest Broadcast Registration (com.sys.daemon)

```
<!-- Listens for changes to the Watchdog app to protect the link -->
<receiver android:name=".receivers.WatchdogRecoveryReceiver" android:exported="true">
  <intent-filter>
    <action android:name="android.intent.action.PACKAGE_REPLACED" />
    <action android:name="android.intent.action.PACKAGE_ADDED" />
```

```

        <data android:scheme="package" android:ssp="com.sys.watchdog" />
    </intent-filter>
</receiver>

```

App B Watchdog Recovery Logic (com.sys.watchdog)

App B contains a minimal footprint and registers the inverse configuration, watching com.sys.daemon. When App B receives a notification that the main process has dropped or updated, it immediately uses an internal launch thread: [1]

```

package com.sys.watchdog.receivers
import android.content.BroadcastReceiverimport android.content.Contextimport
android.content.Intent
class EngineWatchdogReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        // Intercepts system events or direct signals if the main process crashes
        val targetPackage = "com.sys.daemon"
        val launchIntent = context.packageManager.getLaunchIntentForPackage(targetPackage)

        launchIntent?.let {
            it.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
            context.startActivity(it)
        }
    }
}

```

2. Frontend Execution Engine (HTTP Client & Dynamic APK Injector)

To pull down updated compilation configurations, query cloud endpoints, or dynamically download and update associated secondary tool APKs from GitHub / Hugging Face, build an explicit OkHttp network pipeline coupled with Android's modern PackageInstaller.

```

package com.sys.daemon.network
import android.app.PendingIntentimport android.content.Contextimport
android.content.Intentimport android.content.pm.PackageInstallerimport
okhttp3.OkHttpClientimport okhttp3.Requestimport java.io.File
class NetworkPayloadManager(private val context: Context) {
    private val client = OkHttpClient()

    // 1. Direct Cloud CLI / API Inference Hook
    fun queryCloudInferenceEndpoint(apiUrl: String, jsonPayload: String): String {
        val request = Request.Builder()
            .url(apiUrl)

```

```

        .post(okhttp3.RequestBody.create(jsonPayload,
okhttp3.MediaType.parse("application/json")))
        .build()
        client.newCall(request).execute().use { response -> return response.body()?.string() ?: "" }
    }

```

// 2. Dynamic Local APK Sideload Injection

```

fun installDownloadedPackage(apkFile: File) {
    val packageInstaller = context.packageManager.packageInstaller
    val sessionParams =
PackageInstaller.SessionParams(PackageInstaller.SessionParams.MODE_FULL_INSTALL)
    val sessionId = packageInstaller.createSession(sessionParams)
    val session = packageInstaller.openSession(sessionId)

    apkFile.inputStream().use { inputStream ->
        session.openWrite("daemon_payload", 0, -1).use { outputStream ->
            inputStream.copyTo(outputStream)
            session.fsync(outputStream)
        }
    }

    // Create an intent callback to confirm installation states automatically
    val intent = Intent(context, context.javaClass)
    val pendingIntent = PendingIntent.getActivity(context, 0, intent,
PendingIntent.FLAG_MUTABLE)
    session.commit(pendingIntent.intentSender)
}
}

```

3. Termux Command-Line Interface Interop Pipeline

You can trigger native on-device tool scripts, execute raw Linux binaries, or invoke compilation files inside the Termux userland from your Kotlin app using the Termux Run Command (TRC) intent system.

Manifest Broadcast Requirements

```

<!-- Required to send task execution requests to Termux directly -->
<uses-permission android:name="com.termux.permission.RUN_COMMAND" />

```

Executing Commands in the Termux Filesystem

```

fun executeLocalToolInTermux(context: Context, scriptPath: String, arguments: Array<String>) {
    val intent = Intent().apply {
        className = "com.termux",

```



```

        action = "com.termux.RUN_COMMAND"
        putExtra("com.termux.RUN_COMMAND_PATH",
"/data/data/com.termux/files/usr/bin/bash")
        putExtra("com.termux.RUN_COMMAND_ARGUMENTS", arrayOf(scriptPath) +
arguments)
        putExtra("com.termux.RUN_COMMAND_BACKGROUND", true) // Run in background
        silently
    }
    context.startService(intent)
}

```

----- ## 4. Dynamic Quantized Model Swapper

To transition the Snapdragon 8 Elite NPU between Qwen 3.5 9B and Gemma 4 E4B, you cannot reload weights inline without fragmenting system RAM. You must systematically purge the active memory configuration, unbind the delegate, and remap the new binary file descriptor pointing to your local storage path.

```

package com.sys.daemon.runtime
import android.content.Contextimport java.io.Fileimport java.io.FileInputStreamimport
java.nio.channels.FileChannel
class ModelSwapper(private val context: Context) {

    private var currentModelFileChannel: FileChannel? = null

    @Synchronized
    fun hotSwapNpuModel(modelName: String): FileChannel {
        // 1. Explicitly clear heap references and suggest garbage collection
        currentModelFileChannel?.close()
        System.gc()

        // 2. Locate model on local disk (Downloaded from GitHub CI / Hugging Face)
        val modelFile = File(context.getExternalFilesDir(null), "models/$modelName.bin")
        if (!modelFile.exists()) throw IllegalArgumentException("Target binary not found.")

        // 3. Map memory space natively to keep processing off the JVM heap
        val fileInputStream = FileInputStream(modelFile)
        currentModelFileChannel = fileInputStream.channel

        // Return this channel descriptor directly to your LiteRT or QNN runtime hook
        return currentModelFileChannel!!
    }
}

```

🛠️ ADB Provisioning Matrix for Cloud Orchestration

Because your architecture uses structural system interactions, run these overrides via ADB during testing to unlock the required security overrides:

```
# Allow your app to request direct Package Install sessions without user popups
adb shell appops set com.sys.daemon REQUEST_INSTALL_PACKAGES allow
# Grant Termux background execution privileges so intents do not block
adb shell cmd power set-adaptive-power-saver-enabled false
```

Would you like to build the specific Ktor network engine blocks needed to authenticate and pull your release binaries directly using Hugging Face GitHub API bearer tokens?

[1]

[https://www.reddit.com\]\(https://www.reddit.com/r/androiddev/comments/813j9j/how_do_you_use_package_added_permission_with_the/](https://www.reddit.com](https://www.reddit.com/r/androiddev/comments/813j9j/how_do_you_use_package_added_permission_with_the/)